ATM PROJECT

Technical Documentation

Digital design of a Euro cash-withdrawal ATM in VHDL
implemented on a Digilent Basys 3 FPGA board

Sabău Mihai Răzvan

Ilea Ștefan Antoniu

November 10, 2026

Contents

1 Problem statement **3**

2 Black box **3**

3 Inputs and outputs **3**

3.1 Inputs . . . 3

3.2 Outputs . . . 4

3.3 Functional blocks . . . 4

4 Command Unit **5**

5 Execution Unit **6**

6 Design **6**

6.1 MSI components used . . . 6

6.1.1 Modulo-10 counter and Modulo-4 counter . . . 7

6.1.2 DMUX 1:4 on 2 bits . . . 8

6.1.3 16-bit comparator and Comparator1000 . . . 8

6.1.4 SumComparator . . . 9

6.1.5 4-digit reader . . . 10

6.1.6 BCD adder and BCD subtractor . . . 10

6.1.7 Frequency divider and debouncer . . . 12

6.2 The main blocks . . . 12

6.2.1 Verify PIN Block . . . 12

6.2.2 Deposit Block . . . 13

6.2.3 Withdrawal Block . . . 13

6.2.4 Banknotes Memory and Banknotes Needed . . . 14

6.2.5 Cash Handout Block . . . 16

6.2.6 Change PIN Block . . . 17

6.2.7 PIN Memory and Funds Memory . . . 17

6.2.8 7-segment Display Block . . . 18

7 Implementation on the FPGA **20**

8 How to use **21**

8.1 Accounts . . . 22

8.2 Banknotes memory 22
8.3 State diagram 22
8.4 Original state-diagram fragments 22

9 Solution justification **25**

10 Possibilities of future development **26**

A Photos of the running system **27**

1. Problem statement

A bank ATM for EURO cash withdrawals is to be designed. It is assumed that the maximum amount that can be withdrawn at one time is 1,000 euros. Initially, the card is identified, and the operation is selected. The system must support at least 4 different cards/accounts and at least 4 different operations must be implemented.

First, the code of the bank account is checked, and then the desired operation is selected. The supported operations are *Check Balance*, *Deposit*, *Withdrawal* and *Change PIN*. In the case of a withdrawal, the system first checks that the funds exist, then checks the available banknotes, and only then hands out the sum. After the operation is finished, the user can choose either to perform another operation or to return the card.

2. Black box

Figure 1 shows the system as a black box: the inputs are on the left and the outputs on the right.

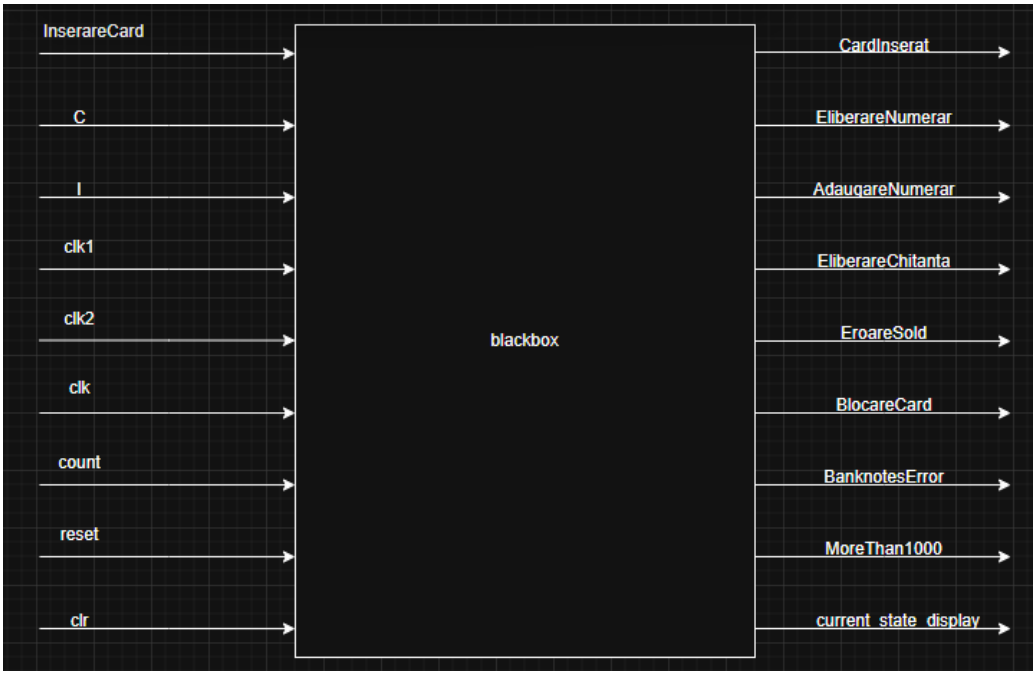


Figure 1: Black box view of the ATM.

3. Inputs and outputs

3.1 Inputs

1. `C` (3 bits) – code of the bank account.
2. `I` (2 bits) – code for operation selection.
3. `clk` – clock signal.
4. `clk1` – main button, used to confirm each operation.

5. `clk2`, `count` – buttons used when numerical data is entered.
6. `updown` – switch choosing the counting direction when numerical data is entered.
7. `InserareCard` – switch that models inserting / removing the card.
8. `reset` – resets the whole system.
9. `clr` – when activated, turns off the 7-segment display without affecting the ATM.

3.2 Outputs

1. `an` / `a_to_g` – control the 7-segment display.
2. `current_state_display` – indicates the state the system is currently in.
3. `CardInserat` – the card is inserted.
4. `EliberareNumerar` – the ATM has successfully handed out the money.
5. `EliberareChitanta` – the receipt is being handed out.
6. `EroareSold` – insufficient funds.
7. `MoreThan1000` – the requested sum is greater than 1000.
8. `BanknotesError` – not enough banknotes.
9. `BlocareCard` – the card is blocked after 3 wrong PIN entries.

3.3 Functional blocks

Command Unit
Controls the entire system and activates the necessary block at the right time. It receives data both from the user and from the other blocks.

Verify PIN Block
Waits for the PIN typed by the user and compares it with the one stored in the memory of the selected account. If the PIN is wrong 3 times, the block blocks the card.

Withdrawal Block
The user enters the desired amount; once it is entered, the block checks whether there are enough funds. If so, the amount is sent to the Handout Amount block.

Handout Amount Block
Knows the number of banknotes available and whether the sum exceeds 1000. If there are enough banknotes and the sum does not exceed 1000, the sum is handed out and the balance is updated.

Deposit Block
Receives an amount from the user and adds it to the current balance of the account.

Change PIN Block
Waits for the new PIN and updates the PIN Memory after the new PIN is confirmed.

PIN Memory
Holds the PIN of each account. It is read by Verify PIN and written by Change PIN.

Funds Memory
Holds the amount of money in each account. It is displayed by Check Balance.

4. Command Unit

The command unit controls the operation of the whole system: it takes decisions and determines the next state, considering the inputs, the data coming from the other blocks, and the PIN and funds memories. It is implemented as a finite state machine (see Section 8.3).

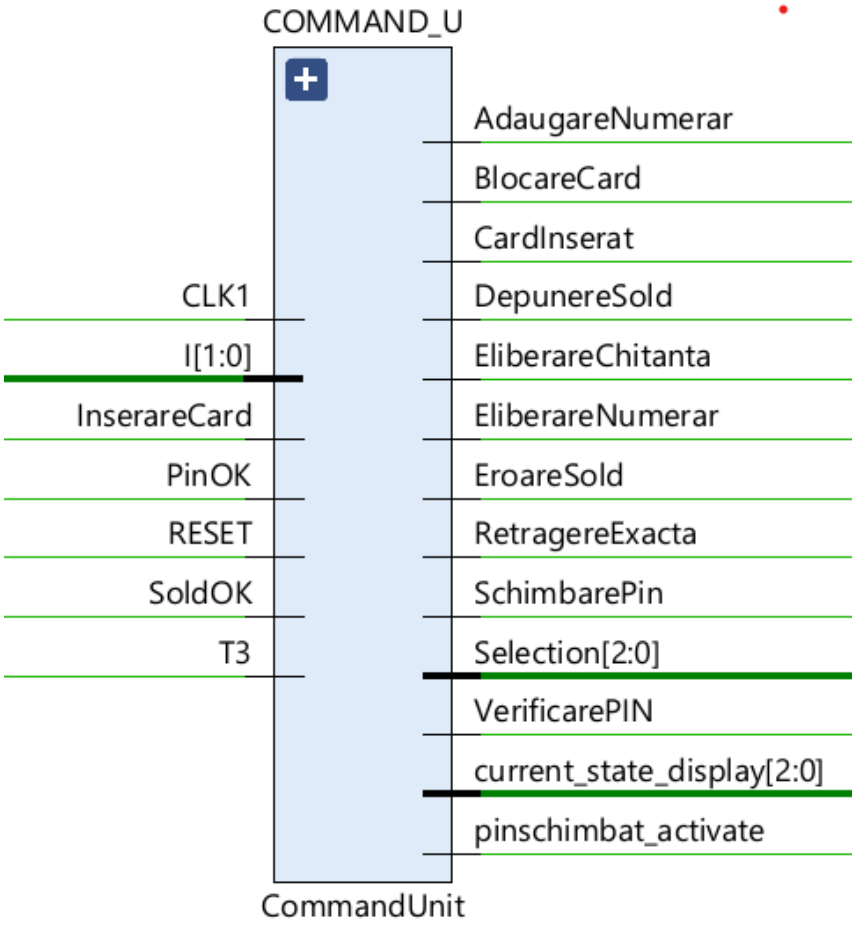


Figure 2: The Command Unit (`CommandUnit`).

Table 1: Ports of the Command Unit.

Inputs	`CLK1`, `I[1:0]`, `InserareCard`, `PinOK`, `RESET`, `SoldOK`, `T3`
Outputs	`AdaugareNumerar`, `BlocareCard`, `CardInserat`, `DepunereSold`, `EliberareChitanta`, `EliberareNumerar`, `EroareSold`, `RetragereExacta`, `SchimbarePin`, `Selection[2:0]`, `VerificarePIN`, `current_state_display[2:0]`, `pinschimbat_activate`

Besides the outputs shown in the diagram, there are a few extra ones. One of them is “Handout Receipt”, which is activated with a delay of one clock period (`CLK1`) after the following conditions

are met simultaneously: the current state is Withdrawal, the input confirms that the user wants a receipt, and `SoldOK` is 1. A D flip-flop is used, whose input is the conjunction of the conditions above; its asynchronous reset is connected to the reset input of the whole system.

The outputs `current_state_display` and `Selection` support the user interface: the first one is routed to the board LEDs, while the second is routed to the Display Block to select the data shown on the 7-segment display.

Two other outputs are `activateb`, which is active in the Withdrawal state and drives the block that processes the amount, and `pinschimbat_activate` ("ChangedPIN" in the diagram). Their VHDL implementation, together with that of `Selection` and `current_state_display`, is shown below.

Listing 1: Output logic of the Command Unit.

```
Selection <= "000" when curr_state = VerificarePinS else
             "001" when curr_state = SchimbarePinS else
             "010" when curr_state = DepunereSoldS else
             "011" when curr_state = RetragereExactaS else
             "100" when curr_state = AdaugareNumerarS else
             "101" when curr_state = EliberareNumerarS else
             "110" when curr_state = InterogareSold else
             "111";

activateb <= '1' when curr_state = RetragereExactaS else
             '0';

pinschimbat_activate <= '1' when curr_state = PinSchimbat else
                        '0';

with curr_state select
  current_state_display <= "000" when Standby,
                           "001" when VerificarePinS,
                           "010" when SelectareOperatiune,
                           "011" when DepunereSoldS,
                           "100" when RetragereExactaS,
                           "101" when SchimbarePinS,
                           "110" when ScoatetiCardul,
                           "111" when others;
```

5. Execution Unit

The execution unit consists of all the components outside the command unit: the Verify PIN Block, Deposit Block, Withdrawal Block, Processing Block, Handout Cash Block, Change PIN Block, PIN Memory and Funds Memory. To these we add a 2:1 MUX, a 16-bit register and a comparator.

6. Design

6.1 MSI components used

Medium Scale Integration (MSI) components were used to ease the design and to be able to provide a scheme for the whole project. They are presented below.

6.1.1 Modulo-10 counter and Modulo-4 counter

The Modulo-10 counter is a 4-bit counter that counts in the interval 0–9 at every input pulse; it counts up if `UPDOWN` is 1 and down if it is 0. The Modulo-4 counter counts in the interval 0–3, only upwards, and has an extra output, `TC` (terminal count), which is active when the counter reaches 3.

Listing 2: Modulo-10 up/down counter.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Modulo10Counter is
  port(
    COUNT  : in  std_logic;
    UPDOWN : in  std_logic;
    RESET  : in  std_logic;
    Q      : out std_logic_vector(3 downto 0)
  );
end Modulo10Counter;

architecture M of Modulo10Counter is
  signal tmp : std_logic_vector(3 downto 0);
begin
  process(COUNT, RESET)
  begin
    if (RESET = '1') then
      tmp <= "0000";
    elsif (COUNT'event and COUNT = '1') then
      if (UPDOWN = '1') then
        if (tmp = "1001") then
          tmp <= "0000";
        else
          tmp <= tmp + 1;
        end if;
      else
        if (tmp = "0000") then
          tmp <= "1001";
        else
          tmp <= tmp - 1;
        end if;
      end if;
    end if;
  end process;
  Q <= tmp;
end M;
```

Listing 3: Modulo-4 counter with terminal count.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Modulo4Counter is
  port(
    UP    : in  std_logic;
    RESET : in  std_logic;
    TC    : out std_logic;  -- terminal count
    Q     : out std_logic_vector(1 downto 0)
  );
end Modulo4Counter;
```

```
architecture M of Modulo4Counter is
  signal tmp : std_logic_vector(1 downto 0);
begin
  process(UP, RESET)
  begin
    if (RESET = '1') then
      tmp <= "00";
    elsif (UP'event and UP = '1') then
      tmp <= tmp + 1;
    end if;
  end process;
  Q  <= tmp;
  TC <= '1' when tmp = "11" else '0';
end M;
```

6.1.2 DMUX 1:4 on 2 bits

This component routes the 2-bit input data to one of its 4 outputs, selected by `sel`; the other outputs are forced to `"00"`.

Listing 4: 2-bit 1:4 demultiplexer.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity dmux21_2bits is
  port(
    input : in  std_logic_vector(1 downto 0);
    sel   : in  std_logic_vector(1 downto 0);
    o0    : out std_logic_vector(1 downto 0);
    o1    : out std_logic_vector(1 downto 0);
    o2    : out std_logic_vector(1 downto 0);
    o3    : out std_logic_vector(1 downto 0)
  );
end dmux21_2bits;

architecture D of dmux21_2bits is
begin
  process(input, sel)
  begin
    if (sel = "00") then
      o0 <= input; o1 <= "00"; o2 <= "00"; o3 <= "00";
    elsif (sel = "01") then
      o1 <= input; o0 <= "00"; o2 <= "00"; o3 <= "00";
    elsif (sel = "10") then
      o2 <= input; o0 <= "00"; o1 <= "00"; o3 <= "00";
    else
      o3 <= input; o0 <= "00"; o1 <= "00"; o2 <= "00";
    end if;
  end process;
end D;
```

6.1.3 16-bit comparator and Comparator1000

The 16-bit comparator is a simple comparator that activates the `EQUALITY` signal when inputs `A` and `B` are equal. `Comparator1000`, on the other hand, compares the input `SumNeeded` with the number 1000 in BCD; if the input is not greater than 1000, the signal `LessThan1000` is

activated. This component prevents withdrawals larger than 1000.

Listing 5: 16-bit equality comparator.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity comparator16bit is
  port(
    A        : in  std_logic_vector(15 downto 0);
    B        : in  std_logic_vector(15 downto 0);
    EQUALITY : out std_logic
  );
end comparator16bit;

architecture C of comparator16bit is
begin
  EQUALITY <= '1' when A = B else '0';
end C;
```

Listing 6: Comparator against 1000 (BCD).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity comparator1000 is
  port(
    SumNeeded    : in  std_logic_vector(15 downto 0);
    LessThan1000 : out std_logic
  );
end comparator1000;

architecture comp of comparator1000 is
begin
  LessThan1000 <= '1' when SumNeeded <= "0001000000000000" else '0';
end comp;
```

6.1.4 SumComparator

This component compares two 16-bit BCD numbers, starting with the most significant digit and moving on to the next digit only when the current ones are equal, until it can decide which number is smaller. If the `Available` number is greater than or equal to the `Needed` one, `SoldOK` becomes 1.

Listing 7: BCD sum comparator.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SumComparator is
  port(
    Available : in  std_logic_vector(15 downto 0);
    Needed    : in  std_logic_vector(15 downto 0);
    SoldOK    : out std_logic
  );
end SumComparator;

architecture S of SumComparator is
begin
  process(Available, Needed)
  begin
```

```
    if (Available(15 downto 12) > Needed(15 downto 12)) then
      SoldOK <= '1';
    elsif (Available(15 downto 12) = Needed(15 downto 12)) then
      if (Available(11 downto 8) > Needed(11 downto 8)) then
        SoldOK <= '1';
      elsif (Available(11 downto 8) = Needed(11 downto 8)) then
        if (Available(7 downto 4) > Needed(7 downto 4)) then
          SoldOK <= '1';
        elsif (Available(7 downto 4) = Needed(7 downto 4)) then
          if (Available(3 downto 0) > Needed(3 downto 0)) then
            SoldOK <= '1';
          elsif (Available(3 downto 0) = Needed(3 downto 0)) then
            SoldOK <= '1';
          else
            SoldOK <= '0';
          end if;
        else
          SoldOK <= '0';
        end if;
      else
        SoldOK <= '0';
      end if;
    else
      SoldOK <= '0';
    end if;
  end process;
end S;
```

6.1.5 4-digit reader

This component allows numbers to be entered in BCD. It receives data from the user and memorises it.

- It has two input signals, `CLK2` and `COUNT`, each with its own purpose. The output is a 16-bit number.
- At the starting moment (after the last reset of the system) all 4 digits are 0 and the selected digit is the leftmost one.
- In this state the user can modify the value of the selected digit using the `COUNT` button and the `UPDOWN` switch: at every `COUNT` press the digit increases or decreases, depending on `UPDOWN`.
- When `CLK2` is pressed, the selected position moves one place to the right. From the rightmost position it wraps back to the first one.
- The component is built from one Modulo-4 counter (position), four Modulo-10 counters (the value of each digit is stored in one of them) and a 1:4 DMUX.

6.1.6 BCD adder and BCD subtractor

These components perform addition and subtraction in BCD. A component handling a single digit (4 bits) was created first; it can then be cascaded for several digits using the `CIN` and `COUT` ports.

Listing 8: Single-digit BCD adder.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity bcd_adder_digit is
  port(
    A    : in  std_logic_vector(3 downto 0);
    B    : in  std_logic_vector(3 downto 0);
    CIN  : in  std_logic_vector(3 downto 0);
    SUM  : out std_logic_vector(3 downto 0);
    COUT : out std_logic_vector(3 downto 0)
  );
end bcd_adder_digit;

architecture bcd of bcd_adder_digit is
  signal tmp : std_logic_vector(3 downto 0);
begin
  tmp <= A + B + CIN;
  process(tmp, A, B, CIN)
  begin
    if (tmp > 9) then
      SUM  <= tmp + 6;
      COUT <= "0001";
    else
      SUM  <= tmp;
      COUT <= "0000";
    end if;
  end process;
end bcd;
```

Listing 9: Single-digit BCD subtractor.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity bcd_substractor_digit is
  port(
    A    : in  std_logic_vector(3 downto 0);
    B    : in  std_logic_vector(3 downto 0);
    CIN  : in  std_logic_vector(3 downto 0);
    DIFF : out std_logic_vector(3 downto 0);
    COUT : out std_logic_vector(3 downto 0)
  );
end bcd_substractor_digit;

architecture D of bcd_substractor_digit is
  signal tmp : std_logic_vector(3 downto 0);
begin
  tmp <= B + CIN;
  process(A, B, tmp)
  begin
    if (A < tmp) then
      DIFF <= "1010" - (tmp - A);
      COUT <= "0001";
    else
      DIFF <= A - tmp;
      COUT <= "0000";
    end if;
  end process;
end D;
```

6.1.7 Frequency divider and debouncer

These two components are essential. The divider reduces the internal clock frequency, which is needed to build the debouncer.

The purpose of the debouncer is to prevent button bounce (the transmission of false information to the system when a button is pressed). It works on the divided clock: the button must be seen active on 3 consecutive divided-clock periods for the signal to be accepted. It is built from 3 D flip-flops and an AND gate.

With the 100 MHz clock of the Basys 3 board, dividing by 500 000 yields an enable pulse with a frequency of 200 Hz.

Listing 10: Frequency divider.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity freq_divider is
  port(
    CLK  : in  std_logic;
    CLK0 : out std_logic
  );
end freq_divider;

architecture freq_divider_behavioral of freq_divider is
  signal count : INTEGER range 0 to 499999 := 0;
begin
  Divide : process (CLK)
  begin
    if (CLK'event and CLK = '1') then
      if (count = 499999) then
        CLK0  <= '1';
        count <= 0;
      else
        count <= count + 1;
        CLK0  <= '0';
      end if;
    end if;
  end process Divide;
end freq_divider_behavioral;
```

6.2 The main blocks

The list of the main blocks was presented above. Having presented the command unit, we now present the internal structure of the remaining blocks.

Each block contains a `RESET`. Therefore, while a block is not needed, it is held in the reset state until the Command Unit signals that it has to come into action.

6.2.1 Verify PIN Block

This block waits for a 4-digit input from the user, representing the PIN to be verified. The code is received through the 4-digit reader and compared with the PIN stored in memory using a 16-bit comparator. Entering the PIN wrongly 3 times leads to blocking the card; a Modulo-4

counter is used for this purpose.

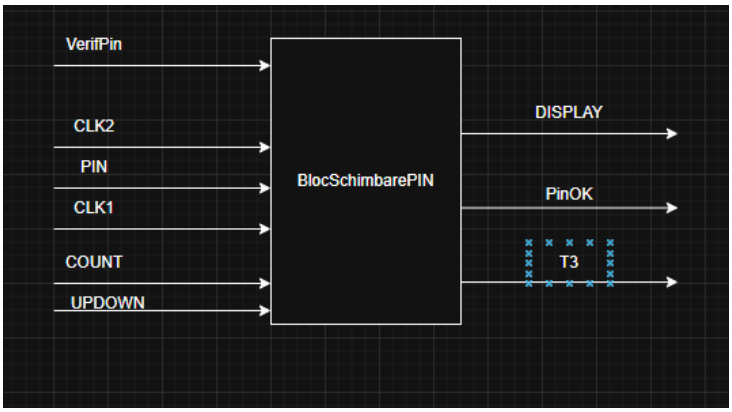


Figure 3: Verify PIN block.

6.2.2 Deposit Block

This block is used to deposit a certain amount of money into the selected account. A 4-digit reader lets the user enter the desired sum, and its output is sent to the adder block (Figure 5), which computes the new balance that is then written into the memory holding the balances.

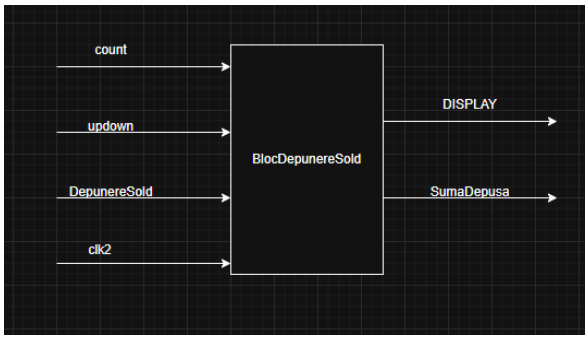


Figure 4: Deposit block.

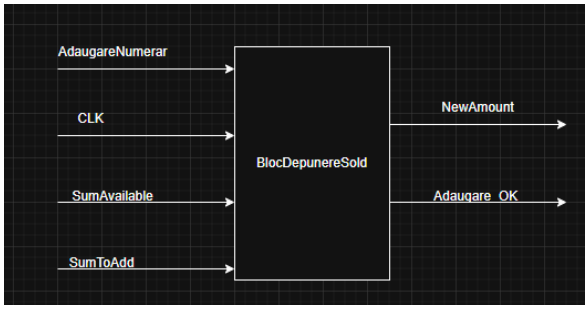


Figure 5: Add-cash block (adds the deposited sum to the balance).

6.2.3 Withdrawal Block

This block uses the 4-digit reader to let the user enter the desired amount of money. It then uses the SumComparator to check whether the amount is available in the account. The result is shown on the 7-segment display.

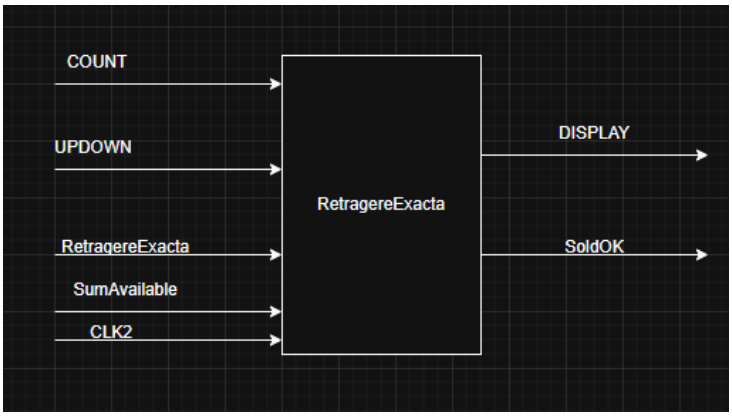


Figure 6: Withdrawal block.

6.2.4 Banknotes Memory and Banknotes Needed

The ATM has a reserve of banknotes. For this, an 8×8-bit RAM was created, holding the available quantity of each banknote (BCD) with values 5, 10, 20, 50, 100, 200 and 500. The VHDL code is:

Listing 11: Banknotes memory.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity MagazieBancnote is
  port(
    CLK    : in  std_logic;
    SEL    : in  std_logic_vector(2 downto 0);
    DIN    : in  std_logic_vector(7 downto 0);
    WE     : in  std_logic;
    OUTPUT : out std_logic_vector(7 downto 0)
  );
end MagazieBancnote;

architecture A1 of MagazieBancnote is
  type T is array(7 downto 0) of std_logic_vector(7 downto 0);
  signal content : T := ("00000000", "00000001", "10000101", "00100010",
                         "01010011", "10010010", "10000101", "10010110");
begin
  process(CLK)
  begin
    if (CLK'event and CLK = '1') then
      if (WE = '1') then
        content(conv_integer(SEL)) <= DIN;
      end if;
    end if;
  end process;
  OUTPUT <= content(conv_integer(SEL));
end A1;
```

The *Banknotes Verifier* component is built from:

- Banknotes Memory and Banknotes Needed,
- a 4-digit BCD adder,
- a 2-digit BCD subtractor,

- an 8:1 MUX with 16-bit inputs/outputs (available banknotes) and a 2:1 MUX with 2-bit inputs/outputs.

The algorithm receives as input the amount of money to be extracted. Its purpose is to compute how many banknotes, and of what type, are needed to extract the sum. It uses the clock signal of the system, since it repeats a series of numerical operations.

The algorithm is based on the following registers:

`reg`
16 bits, a BCD number to which the existing banknotes are added step by step, trying to reach the wanted sum. Initially it is `0000`.

`sel`
3 bits, the selection signal sent to the 8:1 MUX.

`OK`
1 bit, signals that the transaction was done successfully: the entered amount is correct and there are enough banknotes.

`write_select`
1 bit, the selection signal of the 2:1 MUX; it indicates which memory is written: Banknotes Memory or Banknotes Needed.

`tmp`
8 bits, the register that counts how many banknotes of a given type are needed.

Listing 12: Write-enable selection and banknote values.

```
process(write_select)
begin
  if (write_select = '0') then
    we_magazie <= '0';
    we_needed  <= '1';
  else
    we_magazie <= '1';
    we_needed  <= '0';
  end if;
end process;

with sel select
  banknote <= "0000010100000000" when "000",  -- 500
              "0000001000000000" when "001",  -- 200
              "0000000100000000" when "010",  -- 100
              "0000000001010000" when "011",  -- 50
              "0000000000100000" when "100",  -- 20
              "0000000000010000" when "101",  -- 10
              "0000000000000101" when "110",  -- 5
              "0000000000000000" when others;
```

The algorithm.

1. The registers are initialised with 0.
2. The algorithm starts with the 500 banknote (`sel` = `"000"`).
3. If banknotes of the current type are still available in memory and `reg` + banknote is smaller than or equal to the sum, then:

   a) `reg` ← `reg` + banknote;
   b) `tmp` ← `tmp` + 1;

   otherwise `write_select` updates Banknotes Needed with `tmp`, then `sel` ← `sel` + 1 and `tmp` ← 0.

4. The previous step is repeated with the next banknote.
5. When `reg` = sum:
   a) `OK` ← 1;
   b) `write_select` ← 1 (to update the Banknotes Memory);
   c) `sel` ← `"000"`;
   d) `OK` ← 0.

For this algorithm to work, the reset input must have been activated at least once beforehand; a new reset is needed for each new use. The algorithm also needs a clock signal; in this case the FPGA clock is used. Auxiliary signals:

- `din_BanknotesNeeded`: input for writing Banknotes Needed;
- `SumNeeded`: the sum that has to be extracted;
- `to_add`: `reg` plus the banknote being verified.

6.2.5 Cash Handout Block

Using the Banknotes Verifier, this block checks that the necessary banknotes exist. If they do, it compares the sum in the account with the sum requested by the user and updates the Banknotes Memory. Otherwise, it activates the `BanknotesError` signal.

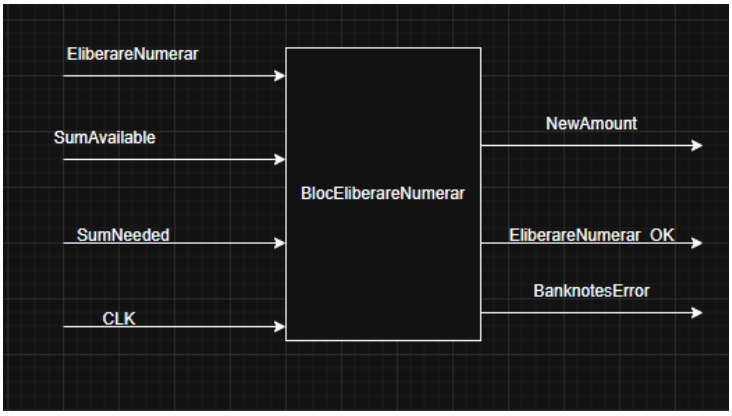


Figure 7: Cash handout block.

After checking that the user has the money in the account and that the banknotes are sufficient, the block checks that the amount does not exceed 1000, using `Comparator1000`.

Because this block changes the content of the Funds Memory, the existing sum must be saved in a register to avoid functioning problems. The relevant signals are:

- `current_amount`: content of the register;
- `withdrawal_signal`: identical to the withdrawal state of the CU;
- `monmemout_signal`: content of the Funds Memory for the selected account.

6.2.6 Change PIN Block

This block has access to the PIN Memory. It waits for the new PIN from the user, which comes through the 4-digit reader, and overwrites the old PIN with it.

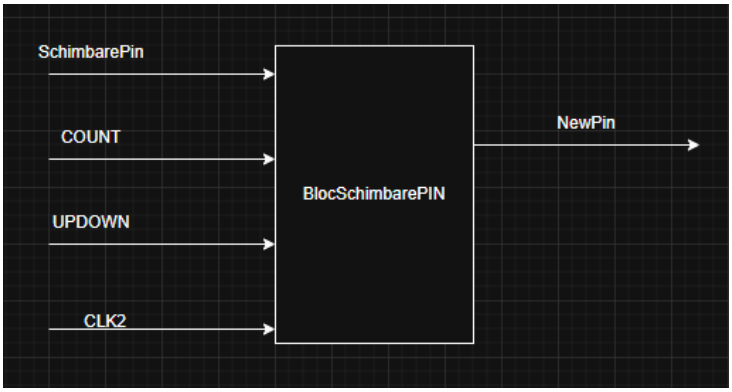


Figure 8: Change PIN block.

6.2.7 PIN Memory and Funds Memory

These two RAM memories, both with 8 addresses of 16 bits each, contain the PINs and the amount of money of each user.

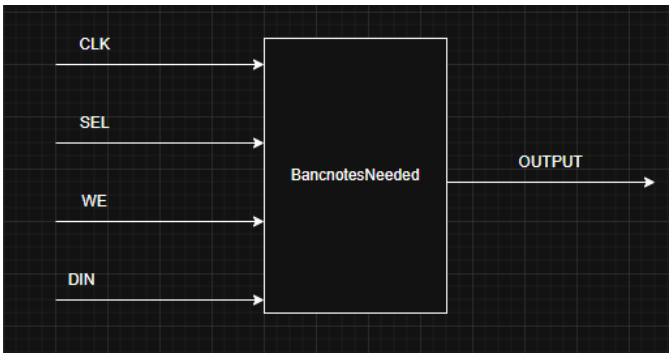


Figure 9: Common interface of the memory blocks (`CLK`, `SEL`, `WE`, `DIN` → `OUTPUT`).

Listing 13: PIN memory.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity PinMemory is
  port(
    CLK    : in  std_logic;
    SEL    : in  std_logic_vector(2 downto 0);
    DIN    : in  std_logic_vector(15 downto 0);
    OUTPUT : out std_logic_vector(15 downto 0)
  );
end PinMemory;

architecture A1 of PinMemory is
  type T is array(7 downto 0) of std_logic_vector(15 downto 0);
  signal content : T := ("0110001100110110", "0100001101110111",
```

```
                        "0101011110000011", "0010010001010001",
                        "0111011100100100", "0101000110000100",
                        "0011001000100000", "0101100001100001");
begin
  process(CLK)
  begin
    if (CLK'event and CLK = '1') then
      content(conv_integer(SEL)) <= DIN;
    end if;
  end process;
  OUTPUT <= content(conv_integer(SEL));
end A1;
```

Listing 14: Funds (money) memory.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity MoneyMemory is
  port(
    CLK    : in  std_logic;
    SEL    : in  std_logic_vector(2 downto 0);
    DIN    : in  std_logic_vector(15 downto 0);
    OUTPUT : out std_logic_vector(15 downto 0)
  );
end MoneyMemory;

architecture A1 of MoneyMemory is
  type T is array(7 downto 0) of std_logic_vector(15 downto 0);
  signal content : T := ("0101011101111001", "1001000000101001",
                         "0110010101110011", "0101011101010001",
                         "0010010101100010", "0001010110000100",
                         "0010000100100011", "0001000001010110");
begin
  process(CLK)
  begin
    if (CLK'event and CLK = '1') then
      content(conv_integer(SEL)) <= DIN;
    end if;
  end process;
  OUTPUT <= content(conv_integer(SEL));
end A1;
```

6.2.8 7-segment Display Block

This block displays all the data we want to show. It receives 16-bit data from several blocks and selects one of them through an 8:1 MUX, controlled by the `Selection` signal generated by the Command Unit. The inputs of the MUX are: Verify PIN, Change PIN, Deposit, Withdrawal, Add Cash, Handout Cash and Check Balance; the last input is `"0000000000000000"`.

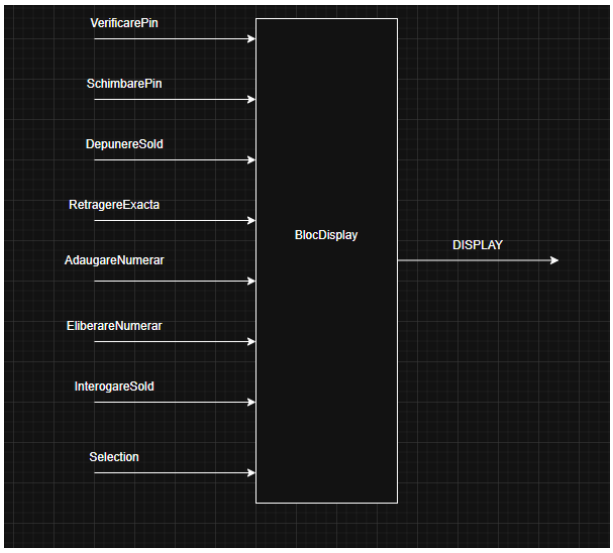


Figure 10: Display block.

Listing 15: Display data selection.

```
with Selection select
  DISPLAY <= VerificarePin    when "000",
             SchimbarePin     when "001",
             DepunereSold     when "010",
             RetragereExacta  when "011",
             AdaugareNumerar  when "100",
             EliberareNumerar when "101",
             InterogareSold   when "110",
             "0000000000000000" when others;
```

BCD to 7-segment conversion. The segments are active-low and ordered `ABCDEFG`.

Table 2: BCD conversion table for the 7-segment display.

B3 B2 B1 B0	A B C D E F G	B3 B2 B1 B0	A B C D E F G
0000	0000001	0101	0100100
0001	1001111	0110	0100000
0010	0010010	0111	0001111
0011	0000110	1000	0000000
0100	1001100	1001	0000100

Listing 16: 7-segment decoder.

```
process(digit)
begin
  case digit is
    when "0000" => a_to_g <= "0000001"; -- '0'
    when "0001" => a_to_g <= "1001111"; -- '1'
    when "0010" => a_to_g <= "0010010"; -- '2'
    when "0011" => a_to_g <= "0000110"; -- '3'
    when "0100" => a_to_g <= "1001100"; -- '4'
    when "0101" => a_to_g <= "0100100"; -- '5'
    when "0110" => a_to_g <= "0100000"; -- '6'
    when "0111" => a_to_g <= "0001111"; -- '7'
```

```
    when "1000" => a_to_g <= "0000000"; -- '8'
    when "1001" => a_to_g <= "0000100"; -- '9'
    when others => a_to_g <= "1111111";
  end case;
end process;
```

7. Implementation on the FPGA

The design was implemented on a Digilent Basys 3 board (Xilinx Artix-7). The pin assignments are listed below.

Table 3: Input pin assignment.

Type	Signal	Pin	Description
Clock	CLK	W5	system clock
Switches	InsertCard	R2	card inserted / removed
	I(1)	T1	operation select
	I(0)	U1	operation select
	C(2)	W2	account code
	C(1)	R3	account code
	C(0)	T2	account code
	UPDOWN	V17	count direction
	RESET	V16	system reset
	clr	W16	display off
Buttons	CLK1	U18	confirm / change state
	CLK2	T17	switch digit (cathodes)
	COUNT	T18	increment / decrement digit

Table 4: Output (LED) pin assignment.

Signal	Pin
CardInserted	L1
Handout Cash	P1
Handout Receipt	N3
ErrorSold	P3
BanknotesError	U3
MoreThan1000	W3
BlockCard	V3
current_state_display(2)	U19
current_state_display(1)	E19
current_state_display(0)	U16

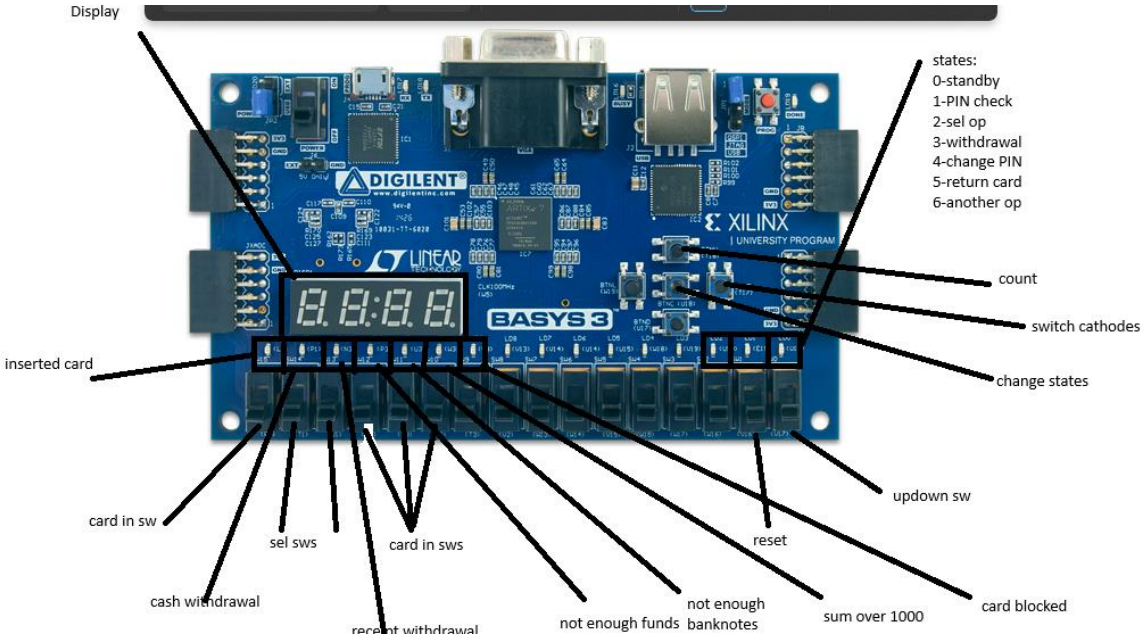


Figure 11: Basys 3 board with the function of each switch, button, LED and the 7-segment display.

8. How to use

1. The ATM starts in *standby*, waiting for a card. The card is inserted with the `InsertCard` switch, together with the code of the desired account (`C`).
2. After the account is chosen, the system waits for the PIN. `CLK1` is pressed to confirm it; if the PIN is correct, the system goes to operation selection. Three wrong PINs block the card.
3. The operation is selected with the `I` switches (Table 5).
4. **Deposit**: the amount is entered with the buttons and confirmed; the system then updates the balance and displays it.
5. **Withdrawal**: the system checks that the sum is available in the account, that the banknotes are sufficient and that the sum does not exceed 1000. If all conditions are met, it hands out the money and updates the balance.
6. **Check Balance**: the user sees the available amount in the account.
7. **Change PIN**: the user enters the new PIN and confirms it.
8. After any operation, the user can either return the card or perform another operation. All states are shown in the diagram of Section 8.3.

Table 5: Operation selection (`I(1)I(0)`), as read from the state diagram.

I(1)	I(0)	Operation
0	0	Deposit
0	1	Withdrawal
1	0	Check Balance
1	1	Change PIN

8.1 Accounts

Table 6: Initial content of the PIN Memory and Funds Memory.

No.	Code	PIN	Balance (EUR)
1	000	5861	1056
2	001	3220	2123
3	010	5184	1584
4	011	7724	2562
5	100	2451	5751
6	101	5783	6573
7	110	4377	9029
8	111	6336	5779

8.2 Banknotes memory

Table 7: Initial quantity of each banknote.

Banknote (EUR)	Quantity
5	1
10	85
20	22
50	53
100	92
200	85
500	96

8.3 State diagram

The flow of the ATM is modelled by the finite state machine below (redrawn from the original diagram). Ellipses are states, diamonds are decisions and rectangles are data-entry steps. In the original drawing the “another operation?” decision is repeated after each operation for layout reasons; here the branches are merged into a single decision.

8.4 Original state-diagram fragments

For reference, the four fragments of the original hand-drawn diagram are reproduced below.

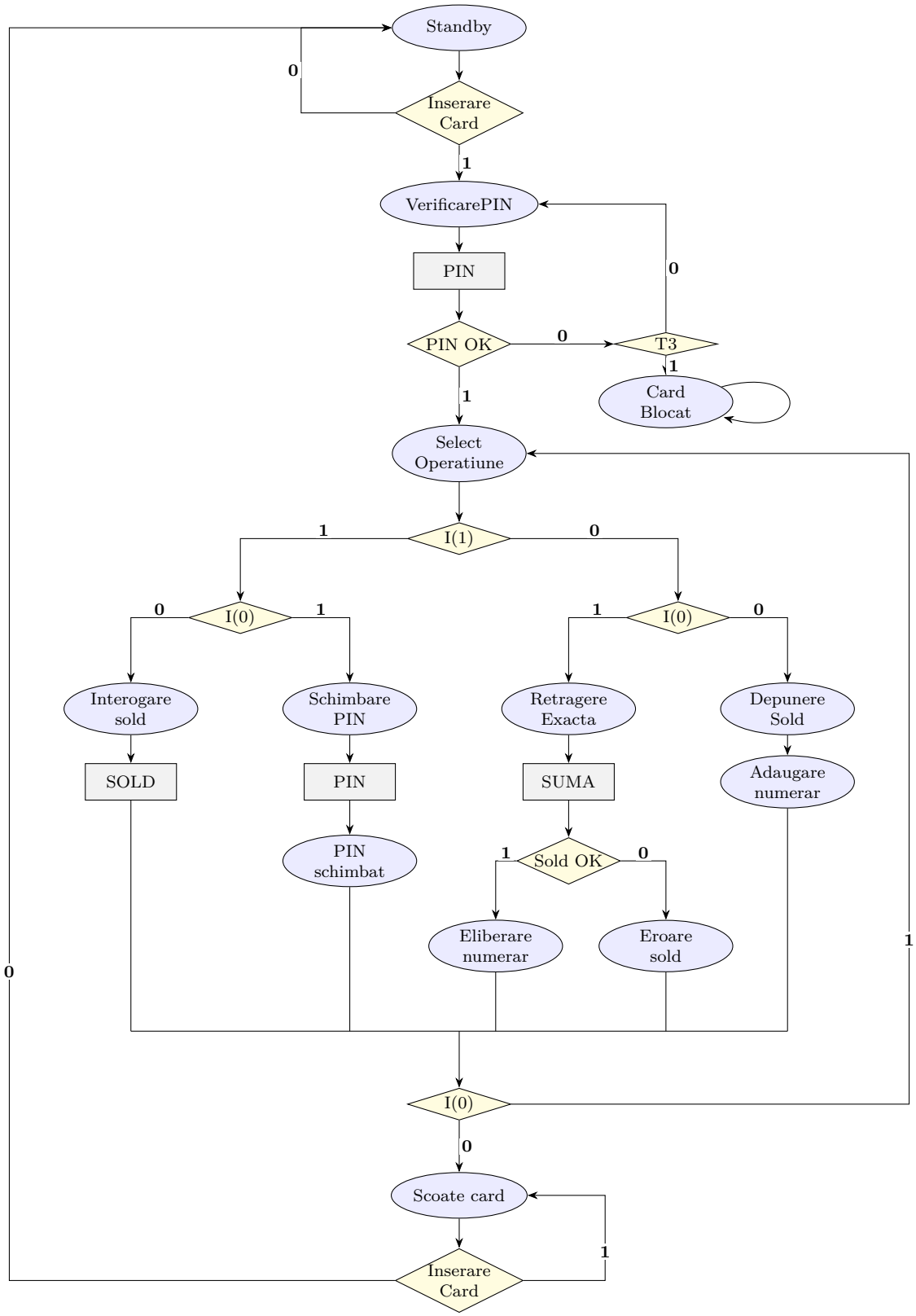


Figure 12: State diagram of the ATM control unit.

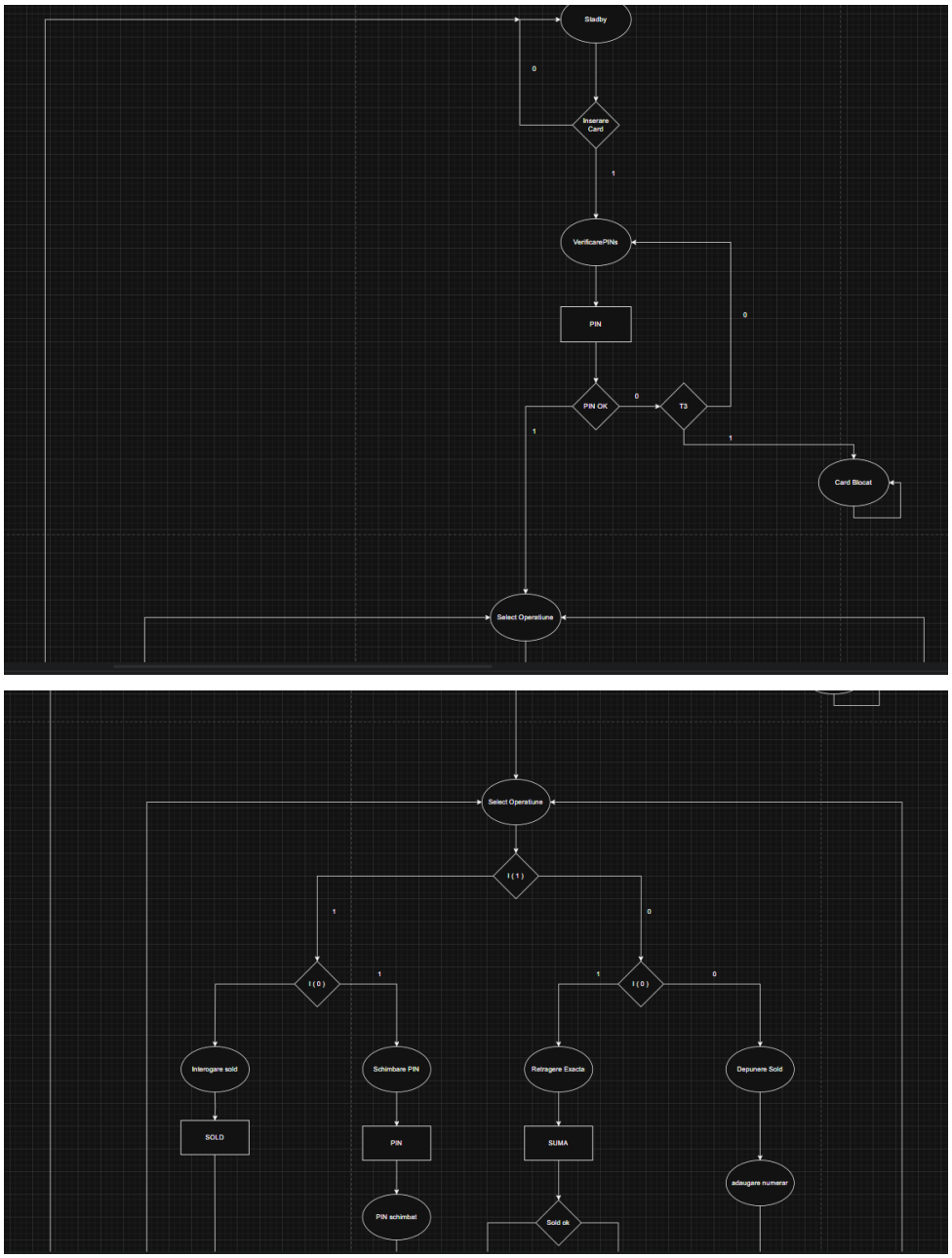


Figure 13: Original state diagram, fragments 1–2.

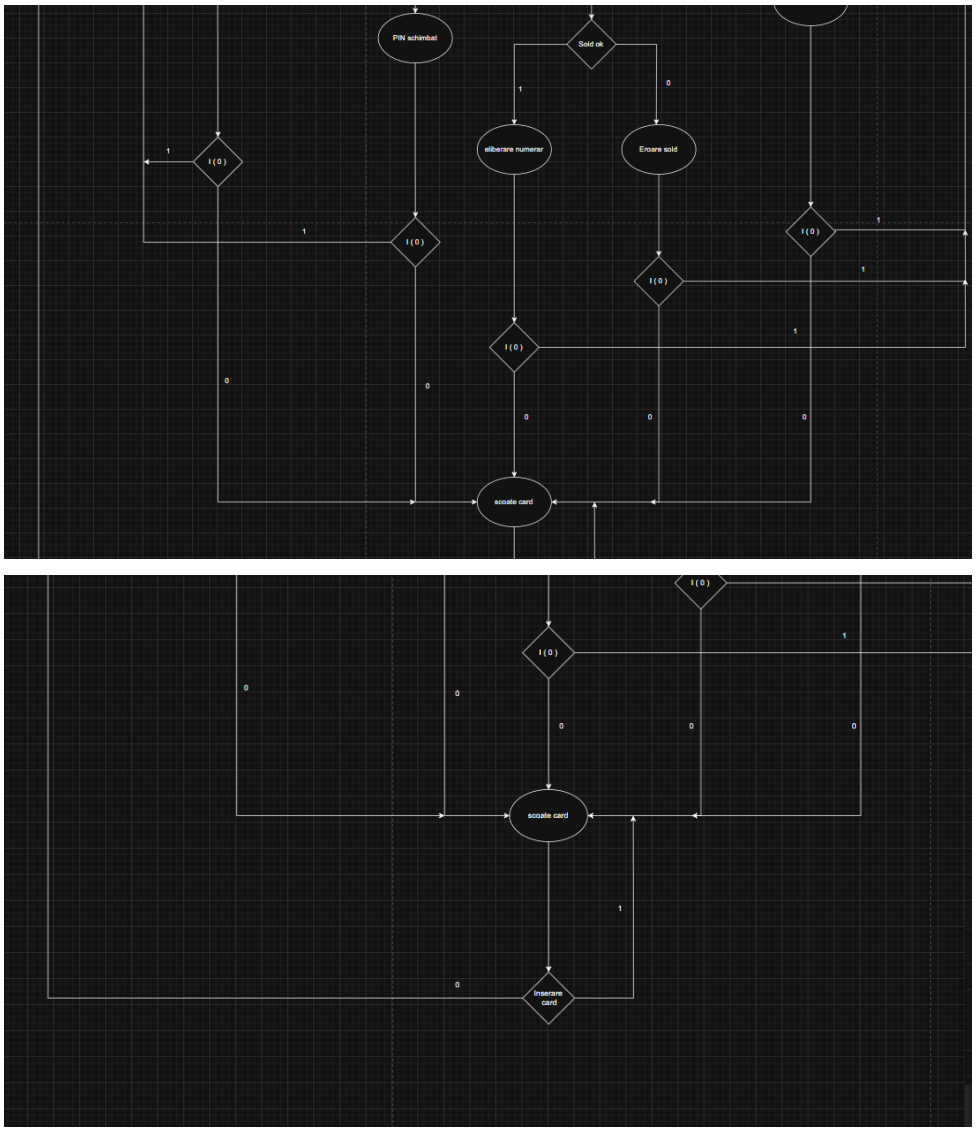


Figure 14: Original state diagram, fragments 3–4.

9. Solution justification

We chose to use an FSM (Finite State Machine) within the Control Unit (CU) because it allows a clear modelling of the control flow specific to an ATM (PIN entry, operation selection, transaction confirmation, etc.), ensuring a scalable and easily testable implementation.

From a functional standpoint, we selected this solution in order to implement the most common operations typically available in ATMs. Additionally, this approach allows handling the errors that generally occur in an ATM system (e.g. incorrect PIN entry).

Another important aspect is the simplicity of the system: elements like the PIN are compared directly with values predefined in components (e.g. the entered PIN is compared with the one assigned to the corresponding account).

Moreover, our solution enables the display of many system states and intermediate results on 7-segment displays or LEDs.

10. Possibilities of future development

1. Encryption of the data in the ATM for enhanced security.
2. Connecting it to a database with many clients, making it a practical tool for the real world.
3. Adding biometric recognition.
4. Improving user interaction by replacing the 7-segment display with a better display.

A. Photos of the running system

The following pictures show the system working on the Basys 3 board.

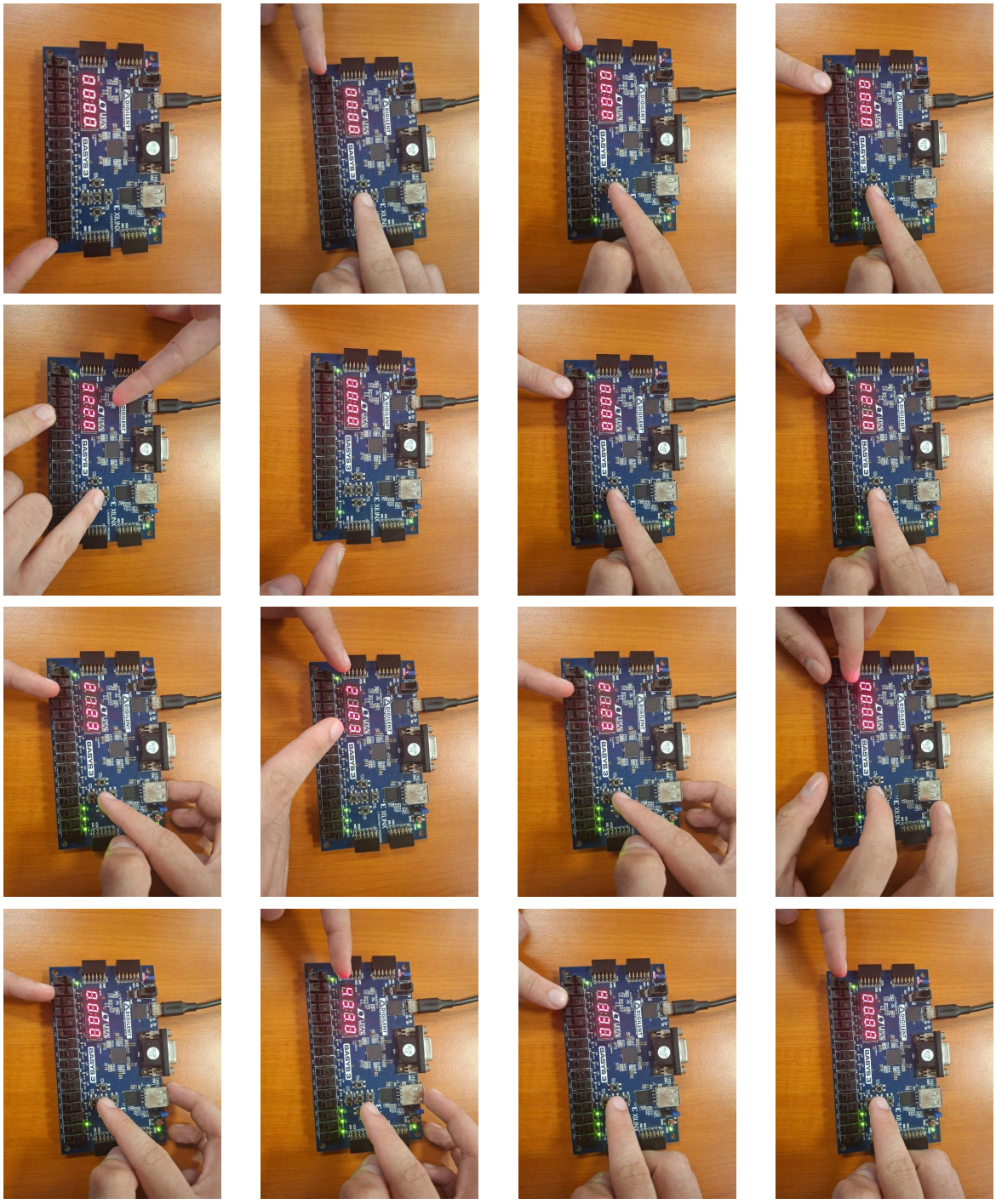


Figure 15: Photos 1–16 of the system running on the board.

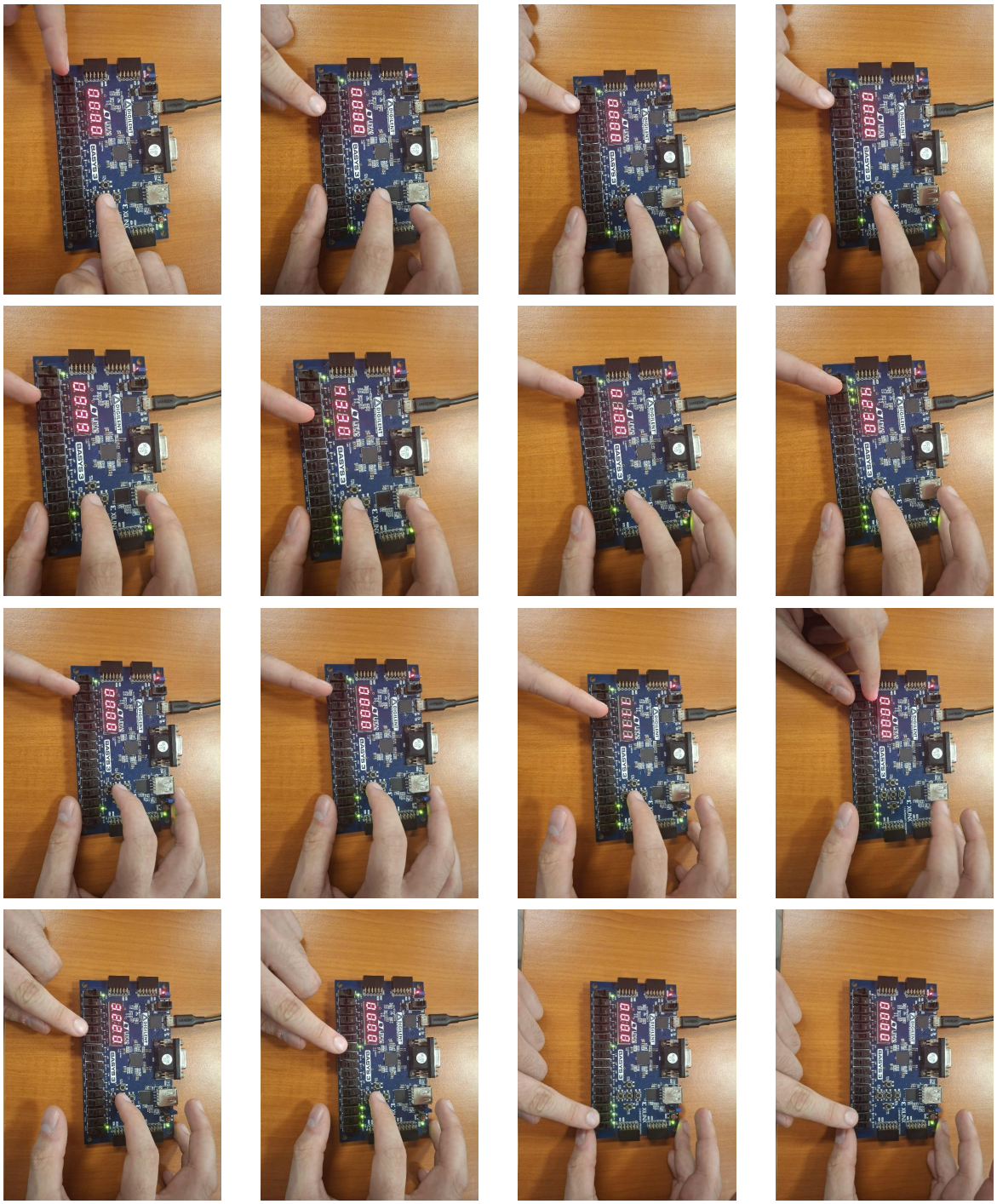


Figure 16: Photos 17–32 of the system running on the board.

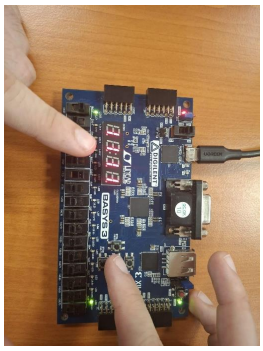




Figure 17: Photos 33–34 of the system running on the board.